Nama: Hans Malvin Djojo

https://github.com/hansmalvin/LastMile

Environment Setup

mongosh
use consistency_lab
db.createCollection("products")

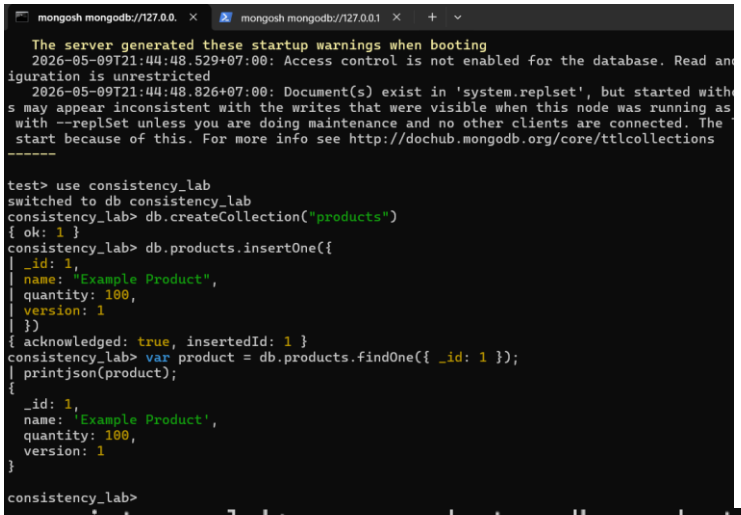
db.products.insertOne({
_id: 1,
name: "Example Product",
quantity: 100,
version: 1
})

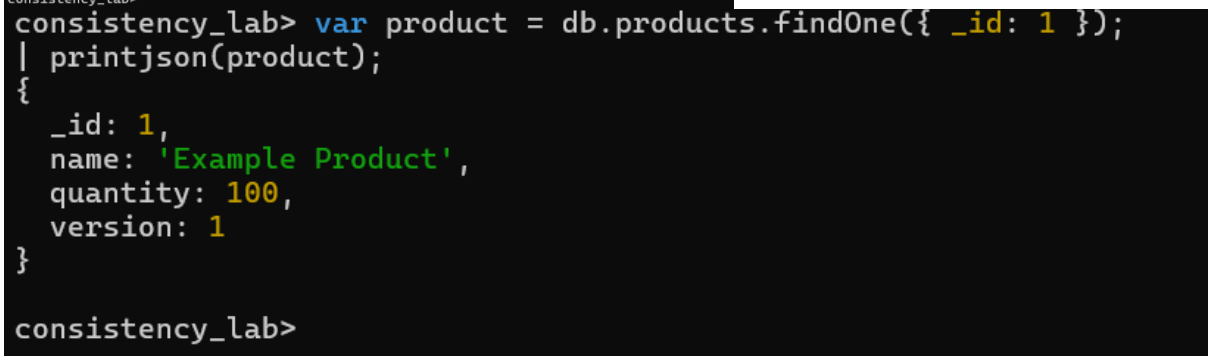
```
test> use consistency_lab
switched to db consistency_lab
consistency_lab> db.createCollection("products")
{ ok: 1 }
consistency_lab> db.products.insertOne({
| _id: 1,
| name: "Example Product",
| quantity: 100,
| version: 1
| })
{ acknowledged: true, insertedId: 1 }
consistency_lab>
```

Step 1: Simulate Concurrent Updates

// Shell 1 & Shell 2
var product = db.products.findOne({ _id: 1 });
printjson(product);

Both shell the same




```
// Shell 1
var currentQuantity = product.quantity;
var newQuantity = currentQuantity - 20;
db.products.updateOne(
{ _id: 1 },
{ $set: { quantity: newQuantity } }
);
var updatedProduct = db.products.findOne({ _id: 1 });
printjson(updatedProduct);
```

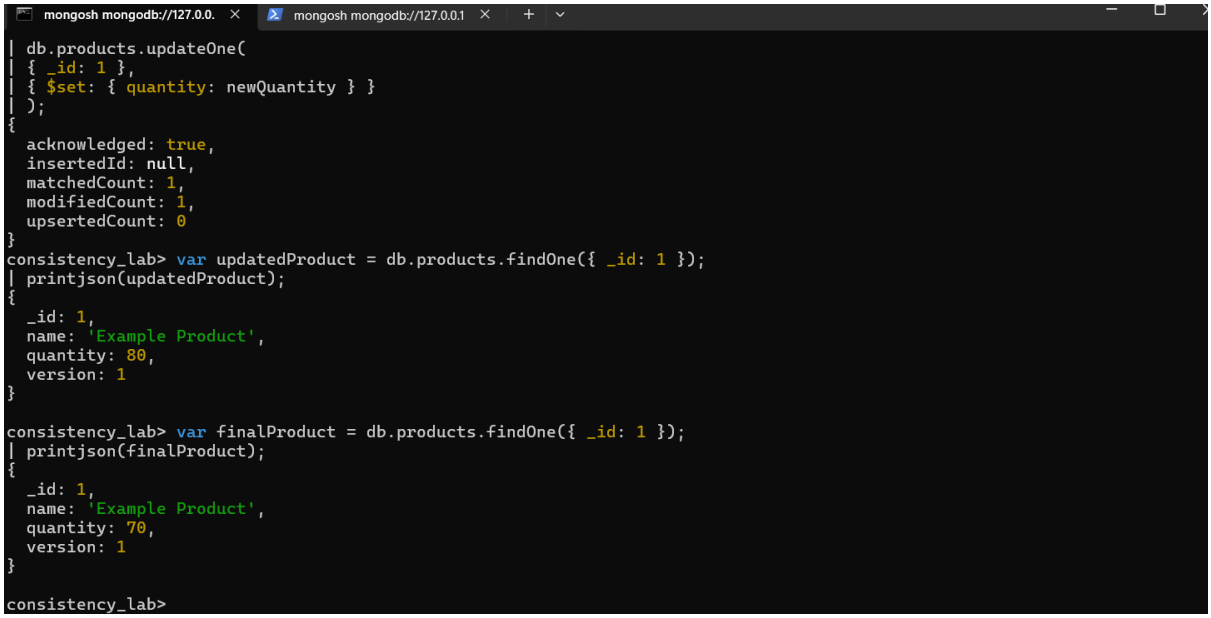
```
consistency_lab> var currentQuantity = product.quantity;
| var newQuantity = currentQuantity - 20;
| db.products.updateOne(
| { _id: 1 },
| { $set: { quantity: newQuantity } }
| );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
consistency_lab> var updatedProduct = db.products.findOne({ _id: 1 });
| printjson(updatedProduct);
{
  _id: 1,
  name: 'Example Product',
  quantity: 80,
  version: 1
}

consistency_lab>
```

```
// Shell 2
var currentQuantity = product.quantity;
var newQuantity = currentQuantity - 30;
db.products.updateOne(
{ _id: 1 },
{ $set: { quantity: newQuantity } }
);
var updatedProduct = db.products.findOne({ _id: 1 });
printjson(updatedProduct);
```

```
consistency_lab> var updatedProduct = db.products.findOne({ _id: 1 });
| printjson(updatedProduct);
{
  _id: 1,
  name: 'Example Product',
  quantity: 70,
  version: 1
}
```

```
// Shell 1 or Shell 2
var finalProduct = db.products.findOne({ _id: 1 });
printjson(finalProduct);
```

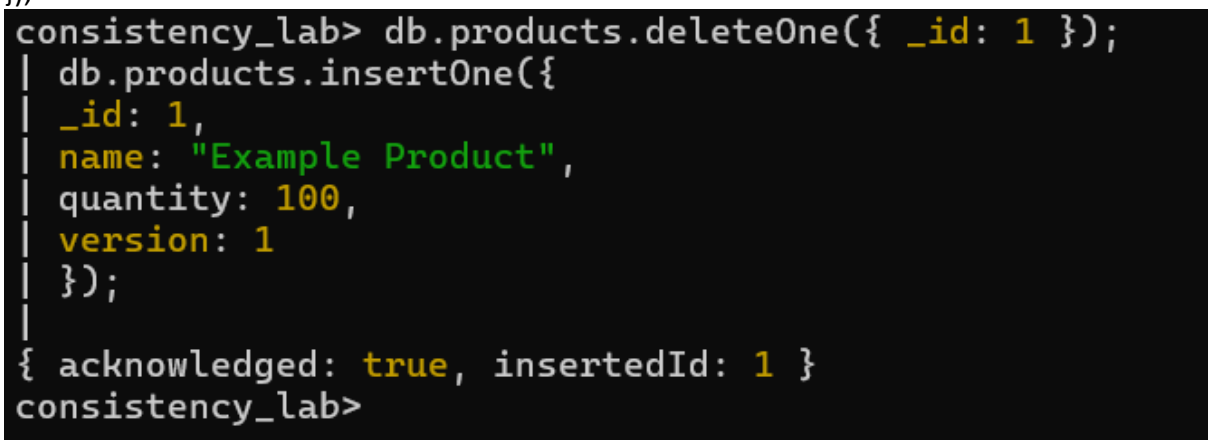



The two is the same (70), from the final quantity that should be 50 but it still 70.

Conclusion: there is an update problem where one update overwrites the other. concurrent updates without proper synchronization lead to data inconsistency.

Step 2: Implement Transactions

db.products.deleteOne({ _id: 1 });
db.products.insertOne({
_id: 1,
name: "Example Product",
quantity: 100,
version: 1
});




// Shell 1 & Shell 2
var session = db.getMongo().startSession();
session.startTransaction();

```
consistency_lab> var session1 = db.getMongo().startSession();
| session.startTransaction();
|

consistency_lab>
consistency_lab> var session2 = db.getMongo().startSession();
| session.startTransaction();
|

consistency_lab>
```

// Shell 1 & Shell 2
var product = db.products.findOne({ _id: 1 }, { session: session1 });
printjson(product);

Shell 1:

```
myReplicaSet [direct: primary] test> use consistency_lab
switched to db consistency_lab
myReplicaSet [direct: primary] consistency_lab> db.createCollection("products")
{ ok: 1 }
myReplicaSet [direct: primary] consistency_lab> db.products.deleteOne({ _id: 1 });
| db.products.insertOne({
| _id: 1,
| name: "Example Product",
| quantity: 100,
| version: 1
| });
|
{ acknowledged: true, insertedId: 1 }
myReplicaSet [direct: primary] consistency_lab> var session = db.getMongo().startSession();
| session.startTransaction();

myReplicaSet [direct: primary] consistency_lab> var sessionDb = session.getDatabase("consistency_lab");

myReplicaSet [direct: primary] consistency_lab> var product = sessionDb.products.findOne({ _id: 1 });
| printjson(product);
{
  _id: 1,
  name: 'Example Product',
  quantity: 100,
  version: 1
}

myReplicaSet [direct: primary] consistency_lab>
```

Shell 2

```
myReplicaSet [direct: primary] test> use consistency_lab
switched to db consistency_lab
myReplicaSet [direct: primary] consistency_lab> db.createCollection("products")
{ ok: 1 }
myReplicaSet [direct: primary] consistency_lab> db.products.deleteOne({ _id: 1 });
| db.products.insertOne({
| _id: 1,
| name: "Example Product",
| quantity: 100,
| version: 1
| });
|
{ acknowledged: true, insertedId: 1 }
myReplicaSet [direct: primary] consistency_lab> var session2 = db.getMongo().startSession();
| session2.startTransaction();

myReplicaSet [direct: primary] consistency_lab> var sessionDb2 = session2.getDatabase("consistency_lab");

myReplicaSet [direct: primary] consistency_lab> var product = sessionDb2.products.findOne({ _id: 1 });
| printjson(product);
{
  _id: 1,
  name: 'Example Product',
  quantity: 100,
  version: 1
}

myReplicaSet [direct: primary] consistency_lab>
```

// Shell 1
var currentQuantity = product.quantity;
var newQuantity = currentQuantity - 20;
db.products.updateOne(
{ _id: 1, $session: session },
{ $set: { quantity: newQuantity } }
);
var updatedProduct = db.products.findOne({ _id: 1, $session: session });
printjson(updatedProduct);

```
myReplicaSet [direct: primary] consistency_lab> var currentQuantity = product.quantity;
| var newQuantity = currentQuantity - 20;
| db.products.updateOne(
| { _id: 1, $session: session },
| { $set: { quantity: newQuantity } }
| );
MongoRuntimeError: ClientSession cannot be serialized to BSON.
myReplicaSet [direct: primary] consistency_lab> var currentQuantity = product.quantity;
| var newQuantity = currentQuantity - 20;

myReplicaSet [direct: primary] consistency_lab> sessionDb.products.updateOne(
|    { _id: 1 },
|    { $set: { quantity: newQuantity } }
| );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
myReplicaSet [direct: primary] consistency_lab>
```

// Shell 2
var currentQuantity = product.quantity;
var newQuantity = currentQuantity - 30;
db.products.updateOne(

{ _id: 1, $session: session },
{ $set: { quantity: newQuantity } }
);
var updatedProduct = db.products.findOne({ _id: 1, $session: session });
printjson(updatedProduct);

```
myReplicaSet [direct: primary] consistency_lab> var currentQuantity = product.quantity;

myReplicaSet [direct: primary] consistency_lab> var newQuantity = currentQuantity - 30;

myReplicaSet [direct: primary] consistency_lab> sessionDb2.products.updateOne(
|   { _id: 1 },
|   { $set: { quantity: newQuantity } }
| );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
myReplicaSet [direct: primary] consistency_lab> var updatedProduct = sessionDb2.products.findOne({ _id: 1 });
| printjson(updatedProduct);
{
  _id: 1,
  name: 'Example Product',
  quantity: 70,
  version: 1
}

myReplicaSet [direct: primary] consistency_lab>
```

// Shell 1
session.commitTransaction();
session.endSession();

```
myReplicaSet [direct: primary] consistency_lab> session.commitTransaction();
| session.endSession();
|

myReplicaSet [direct: primary] consistency_lab>
```

// Shell 2
try {
session.commitTransaction();
session.endSession();
} catch (e) {
print(e);
}

// Shell 1 or Shell 2
var finalProduct = db.products.findOne({ _id: 1 });
printjson(finalProduct);

```
myReplicaSet [direct: primary] consistency_lab> var finalProduct = db.products.findOne({ _id: 1 });
| printjson(finalProduct);
{
  _id: 1,
  name: 'Example Product',
  quantity: 80,
  version: 1
}

myReplicaSet [direct: primary] consistency_lab>
```

Observations: both shell instances were initially able to start separate sessions, read the same starting quantity of 100, and perform local updates within their respective transactions. Although Shell 2 successfully performed its calculation to set the quantity to 70 locally, a conflict was triggered the moment it attempted to commit its transaction after Shell 1 had already finalized its update to 80. This resulted in a WriteConflict error, demonstrating that MongoDB's transaction management tracks document modifications and prevents a session from committing changes based on a version of the data that is no longer current.

Conclusions: implementation of transactions effectively solves the lost update problem by providing a synchronization mechanism that ensures data integrity during concurrent operations. Because Shell 2's conflicting update was rejected upon the detection of the write conflict, its attempt to overwrite Shell 1's work was prevented, leaving the final quantity at a consistent value of 80. This proves that transactions are a reliable way to maintain accuracy in a multi-user environment, as they force conflicting operations to fail rather than allowing them to silently create data inconsistencies.

Step 3: Implement Optimistic Locking

```
db.products.deleteOne({ _id: 1 });
db.products.insertOne({
_id: 1,
name: "Example Product",
quantity: 100,
version: 1
});

// Shell 1 & Shell 2
var product = db.products.findOne({ _id: 1 });
printjson(product);
```

```
myReplicaSet [direct: primary] consistency_lab> db.products.deleteOne({ _id: 1 });
| db.products.insertOne({
| _id: 1,
| name: "Example Product",
| quantity: 100,
| version: 1
| });
|
{ acknowledged: true, insertedId: 1 }
myReplicaSet [direct: primary] consistency_lab> var product = db.products.findOne({ _id: 1 });
| printjson(product);
{
  _id: 1,
  name: 'Example Product',
  quantity: 100,
  version: 1
}

myReplicaSet [direct: primary] consistency_lab>
```

```
// Shell 1
var currentQuantity = product.quantity;
var newQuantity = currentQuantity - 20;
var currentVersion = product.version;
```

```
var newVersion = currentVersion + 1;
var result = db.products.updateOne(
{ _id: 1, version: currentVersion },
{ $set: { quantity: newQuantity, version: newVersion } }
);

printjson(result);

var updatedProduct = db.products.findOne({ _id: 1 });
printjson(updatedProduct);
```

```
myReplicaSet [direct: primary] consistency_lab> var currentQuantity = product.quantity;
| var newQuantity = currentQuantity - 20;
| var currentVersion = product.version;
| var newVersion = currentVersion + 1;
| var result = db.products.updateOne(
| { _id: 1, version: currentVersion },
| { $set: { quantity: newQuantity, version: newVersion } }
| );
|
| printjson(result);
|
| var updatedProduct = db.products.findOne({ _id: 1 });
| printjson(updatedProduct);
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
{
  _id: 1,
  name: 'Example Product',
  quantity: 80,
  version: 2
}

myReplicaSet [direct: primary] consistency_lab>
```

```
// Shell 2
var currentQuantity = product.quantity;
var newQuantity = currentQuantity - 30;
var currentVersion = product.version;
var newVersion = currentVersion + 1;
var result = db.products.updateOne(
{ _id: 1, version: currentVersion },
{ $set: { quantity: newQuantity, version: newVersion } }
);

printjson(result);

var updatedProduct = db.products.findOne({ _id: 1 });
printjson(updatedProduct);
```

```
myReplicaSet [direct: primary] consistency_lab> var currentQuantity = product.quantity;
| var newQuantity = currentQuantity - 30;
| var currentVersion = product.version;
| var newVersion = currentVersion + 1;
| var result = db.products.updateOne(
| { _id: 1, version: currentVersion },
| { $set: { quantity: newQuantity, version: newVersion } }
| );
|
| printjson(result);
|
| var updatedProduct = db.products.findOne({ _id: 1 });
| printjson(updatedProduct);
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
{
  _id: 1,
  name: 'Example Product',
  quantity: 80,
  version: 2
}

myReplicaSet [direct: primary] consistency_lab>
```

// Shell 1 or Shell 2
var finalProduct = db.products.findOne({ _id: 1 });
printjson(finalProduct);

```
myReplicaSet [direct: primary] consistency_lab> var finalProduct = db.products.findOne({ _id: 1 });
| printjson(finalProduct);
{
  _id: 1,
  name: 'Example Product',
  quantity: 80,
  version: 2
}

myReplicaSet [direct: primary] consistency_lab>
```

observations: including a version field in the update criteria, the database could distinguish between current and stale data without needing an active transaction session. While Shell 1 successfully updated the document and incremented the version from 1 to 2, Shell 2's subsequent update attempt resulted in a matchedCount of 0. This occurred because Shell 2's filter specifically looked for version: 1, which no longer existed in the database after Shell 1's operation. Consequently, Shell 2's update failed silently at the database level affecting no documents while the final data remained correctly at the state set by the first successful update.

conclusions: The implementation of optimistic locking proves that data consistency can be maintained by using a version-checking logic that rejects updates based on outdated information. This method prevents the "lost update" problem by ensuring that a write only succeeds if the document has not been modified since it was last read by the client.